1. Complexity Table / Fast Lookup

Algorithm/Data Struct	Time	Space	Notes
Hash search/insert/delete	expected $O(1)$, worst $O(n)$	$O(n)$	α=n/m
BST ops	$O(h)$	$O(n)$	h $\log n$ balanced, n worst
Inorder BST	$\Theta(n)$	$O(h)$	sorted output
Insertion sort	best $\Theta(n)$, worst $\Theta(n^2)$	$O(1)$	stable
Merge sort	$\Theta(n \log n)$	$\Theta(n)$	stable
Quicksort randomized	expected $\Theta(n \log n)$	$O(\log n)$	worst $\Theta(n^2)$
Counting sort	$\Theta(n+k)$	$\Theta(n+k)$	integer keys
Heapify	$O(\log n)$	recursion	assumes children heaps
Build heap	$\Theta(n)$	$O(1)$	bottom-up
Heapsort	$\Theta(n \log n)$	$O(1)$	not stable
PQ insert/extract	$O(\log n)$	$O(n)$	heap
BFS	$\Theta(V+E)$	$\Theta(V)$	unweighted shortest paths
DFS	$\Theta(V+E)$	$\Theta(V)$	topo/cycles/SCC
Topological sort	$\Theta(V+E)$	$\Theta(V)$	DAG only
Kosaraju SCC	$\Theta(V+E)$	$\Theta(V+E)$	2 DFS
Union-Find op	$O(\alpha(n))$	$O(n)$	rank+compression
Kruskal	$O(E \log E)$	$O(V)$	sort + DSU
Prim	$O(E \log V)$	$O(V+E)$	PQ
Dijkstra	$O(E \log V)$	$O(V+E)$	no negative edges
Bellman-Ford	$\Theta(VE)$	$O(V)$	negative edges OK
DAG shortest path	$\Theta(V+E)$	$O(V)$	topo order
Ford-Fulkerson	$O(E\|f^*\|)$	$O(V^2+E)$	integer caps, residual
Edmonds-Karp	$O(VE^2)$	$O(V^2+E)$	BFS residual
Rod cutting	$\Theta(n^2)$	$\Theta(n)$	DP
Matrix chain	$\Theta(n^3)$	$\Theta(n^2)$	DP
LCS	$\Theta(nm)$	$\Theta(nm)$	DP
Knapsack 0/1	$\Theta(nW)$	$\Theta(nW)/\Theta(W)$	pseudo-poly
Coin change	$\Theta(W \cdot \#\text{coins})$	$\Theta(W)$	unbounded

Exam Pattern Matcher

new optimization + choices	DP: state/base/transition/order/answer/proof
local best + exchange	greedy: safe choice + induction
connect all min weight	MST: Kruskal/Prim + cut property
unweighted shortest path	BFS levels
weighted no negative	Dijkstra
negative weights / cycle	Bellman–Ford
dependencies / precedence	DAG + topological order
capacity / matching / disjoint paths	max-flow reduction
expected count	indicators + linearity
online unknown future	competitive ratio vs OPT
min max feasible value	binary search answer + monotone feasible
mutual reachability forbidden	SCC condensation / comp[u]=comp[v]
capacitated assignment	max-flow with integral capacities
tree path/subtree optimum	postorder DP: inside child / through root
tree traversals reconstruct	inorder+post/pre split at root
time-dependent edges	earliest-arrival Dijkstra/events
grid/assignment/paths	bipartite matching / max-flow

Exam Writeup Checklist

Algorithm: input/output, data structs, return value. Correctness: complete invariant/induction/exchange proof, not idea. DP: State, Base, Transition, Order, Answer, Runtime, Space. Bounds: define n, V, E, W; justify loops. Conventions: < vs $\leq$, inclusive/exclusive, 0/1-index, reachable vs global negative cycle.

Proof Bank: Generic Templates

Loop inv. Init true before loop. Maint: one step preserves. Term: invariant + stop condition $\Rightarrow$ answer.

Induction/substitution. Claim upper/lower bound with constants. State IH for all smaller inputs. Substitute IH. Collect terms. Pick constants so inequality holds. Base cases: choose c large enough for all $n < n_0$.

Substitution trap. if $T(n) \leq c\sum_i (a_i n)^p + bn$ and $\sum_i a_i^p = 1$, then $cn^p + bn$ FAILS. No slack absorbs $+bn$. Strengthen: $T(n) \leq cn^p - dn^q$ with lower-order subtraction.

Midterm recurrence. $T(n) = T(4n/5) + T(3n/5) + an$. Guess $\Theta(n^2)$ because $(4/5)^2 + (3/5)^2 = 1$. Lower: $T(n) \geq T(4n/5) + T(3n/5) \geq c((4n/5)^2 + (3n/5)^2) = cn^2$. Upper must be strengthened: assume $T(m) \leq cm^2 - dm$ for all $m < n$. Then $T(n) \leq c(4n/5)^2 - d(4n/5) + c(3n/5)^2 - d(3n/5) + an = cn^2 - 7dn/5 + an \leq cn^2 - dn$ iff $d \geq 5a/2$. Choose d so, then choose c large for base cases.

D&C. Induct on size. Recursive calls solve smaller problems. Combine covers all optimum cases and constructs/chooses correct result.

DP State meaning. Base correct. Induct in fill order. Transition enumerates all valid first/last choices; dependencies already optimal. Optimal solution uses one enumerated choice; every candidate feasible. Therefore $dp[s]$ optimum. Write answer $= dp[\ldots]$.

Parametric search. Guess answer M. Build feasible(M). Prove monotone: if M works then any larger M works (minimization). Binary search smallest feasible M. Correctness = binary search invariant + feasible predicate exact.

Greedy. Safe choice: for optimum O containing current partial solution, if chosen $e \notin O$, exchange $f \in O$ with e without worsening. Then induct.

Proof Bank: Sorting / Hash / DS

BST inorder. Induct on subtree. Left keys $<$ root, right keys $\geq$ root. IH sorts left/right. Left,root,right sorted and complete.

Rank/interval. Store $size = 1 + size(L) + size(R)$. Rank follows one search path; when going right add $size(L)+1$. Thus $O(h)$. $[a,b] = rank_{\leq}(b) - rank_{<}(a)$.

Heapify. Children are heaps. Swap with larger child if violated; parent fixed. Only swapped child may violate; recurse to leaf. Cost height $O(\log n)$.

BuildHeap. Leaves are heaps. Bottom-up: children already heaps, so Heapify fixes subtree. Runtime: nodes height $h \leq n/2^{h+1}$, total $\sum_h O(h)n/2^h = O(n)$.

Sorting lower bound. Comparison sort decision tree needs $\geq n!$ leaves. Worst-case comparisons $\geq$ height $\geq \log_2(n!) = \Omega(n \log n)$.

Quicksort exp. Pair compared at most once. $X_{ij} = 1$ if ranks i, j compared. Compare iff first pivot among ranks $i..j$ is i or j, so $P(X_{ij}) = 2/(j-i+1)$. Sum $O(n \log n)$.

Hashing. Single key: for key x, $X_y = 1[h(y) = h(x)]$. Universal hashing gives $E[X_y] \leq 1/m$, so expected chain collisions $\leq (n-1)/m = \Theta(\alpha)$ and search $O(1+\alpha)$. Total collisions: $X = \sum_{i<j} X_{ij}$. $X_{ij} = 1[h(x_i) = h(x_j)]$, hence $E[X] \leq \binom{n}{2}/m = n(n-1)/(2m) = \Theta(n2/m)$. k-collision: $E[\#k\text{-collisions}] \leq \binom{n}{k}/m^{k-1}$. Markov: $P[X \geq 1] \leq E[X]$.

Hash mini. Length-ℓ keys: pair collision prob $1/m^\ell$, so $E[\#collisions] = \binom{n}{2}/m^\ell$; if $m^\ell \geq \binom{n}{2}$ then $P[\exists collision] \leq 1$. Random q-color vertices: edge mono prob $1/q$, so $E[\#mono] = |E|/q$.

DSU. Union by rank attaches smaller rank under larger. Rank increases only on equal-rank merge. Path compression preserves reps and shortens future paths. With both: m ops in $O(m\alpha(n))$.

Proof Bank: Graph Traversal

BFS. Queue processed in nondecreasing distance. Discover v from u: $d[v] = d[u] + 1$. Shorter path would come from smaller-distance vertex processed earlier, so v would already be discovered.

DFS/topo. Directed cycle iff DFS has back edge to GRAY ancestor. DAG has no back edges. For every edge $u \to v$, $f[u] > f[v]$; decreasing finish time is topo order.

DFS edge facts. Parenthesis theorem: finish intervals disjoint or nested; nested iff descendant. White-path theorem: v descendant of u iff when u discovered, there is a white path $u \rightsquigarrow v$.

SCC/Kosaraju. Condensation graph is DAG. First DFS finish order puts source SCCs late. In reversed graph, DFS from highest finish cannot leave its SCC to an unvisited SCC. Each second-pass DFS outputs one SCC.

SCC reachability pairs. $u \rightsquigarrow v$ and $v \rightsquigarrow u$ iff $comp[u] = comp[v]$. For forbidden pair (u, v) reject iff same component. Correct because SCC is maximal mutual reachability. Runtime $O(V+E)$.

DAG SP/LP Topo order processes every predecessor before vertex. When processing u, $dist[u]$ already final over all paths ending at u; relaxing outgoing edges propagates optimum once. Runtime $\Theta(V+E)$.

Proof Bank: MST / Shortest Paths / Flow

Cut property. Let e be light edge crossing cut respecting A. Take MST $T \supseteq A$ not using e. Add e creates cycle with another crossing edge f. Since $w(e) \leq w(f)$, $T - f + e$ is MST containing $A \cup \{e\}$.

Cycle property. If MST T contains unique heaviest edge e on cycle, remove e; cycle has another edge f reconnecting cut with $w(f) < w(e)$. Then $T - e + f$ cheaper, contradiction.

Kruskal. DSU components define cut respecting chosen edges. Next accepted edge is lightest crossing some component cut. Cut property says safe. Induct until $|V| - 1$ edges.

Prim. S = vertices in tree. Heap chooses lightest edge crossing $(S, V \setminus S)$. Cut property says safe. Add one vertex; repeat.

Dijkstra. S = finalized. Pick outside x with min tentative dist. Any alternative path first crosses from S to outside at (u, v). Since $w \geq 0$, path length $\geq d[u] + w(u,v) \geq d[v] \geq d[x]$. So $d[x]$ final.

Bellman-Ford. After i passes, all shortest paths using $\leq i$ edges are correct. Induct by last edge relaxation. If no negative cycle, shortest simple path has $\leq V - 1$ edges. If pass V improves, reachable negative cycle.

Max-flow. If no augmenting path exists, let S be vertices reachable from s in residual graph. For every edge $u \in S, v \notin S$: $cf(u,v) = 0$, therefore $f(u,v) = c(u,v)$. For every edge $u \notin S, v \in S$: $cf(v,u) = 0$, therefore $f(u,v) = 0$.

Hence net flow crossing cut equals cut capacity: $|f| = c(S,T)$.

Since every flow $\leq$ every cut, flow and cut are both optimal.

Flow cert. Any flow value $\leq$ any cut capacity. Flow value F + cut capacity F proves both optimal.

Bipartite matching flow. Caps 1 force each left/right vertex used at most once. Integral capacities give integral max flow. Unit flow through $l \to r$ is matched pair; every matching gives same-size flow.

Cap assignment flow. Build $s \to$ people cap 1, person $\to$ acceptable item cap 1, item $\to t$ cap capacity. Feasible iff maxflow = #people. Integral flow = assignment; assignment = integral flow.

SP transform. Multiplying every edge weight by $a > 0$ preserves all shortest paths. Adding a to every edge does not: paths with different number of edges get different extra cost.

Resource SP. State graph (v, b) where b = used budget / remaining battery. Add transition only if resource feasible. Run BFS/Dijkstra depending on edge costs. Size $O(VB + EB)$. Correct since every feasible walk corresponds to one state path and vice versa.

2D sorted matrix. Rows/cols sorted. Start top-right. If $A[i,j] > x$, move left; if $A[i,j] < x$, move down. Each step deletes one impossible row/col, so $O(n+m)$, $O(1)$.

Proof Bank: Trees / Exam Reductions

Tree DP. Postorder. Return height best info from children. Best at u is max(best in left, best in right, best crossing/using u). Induct on subtree: any optimal object either lies fully in one child or uses u.

Tree diameter. Let $h(u) = 1 + \max(h_L, h_R)$, empty height -1. $diam(u) = \max(diam_L, diam_R, h_L + h_R + 2)$. Path either inside one child or crosses through u.

Root path. Longest simple path forced to pass through root: $h(root.left) + h(root.right) + 2$ with empty height -1. Compute heights postorder in $O(n)$.

Tree rebuild. Inorder+postorder: root=last(post); split inorder at root; subtree sizes split postorder; recurse. Preorder+inorder: root=first(pre). Pre+post alone not unique.

Unicyclic spanning trees. Connected graph with $|E| = |V|$ has exactly one cycle C. Every spanning tree = delete exactly one edge of C. Deleting non-cycle edge disconnects; deleting cycle edge breaks only cycle. Hence #spanning trees $= |C|$.

Proof Bank: Online / Random

LRU paging. Cache size k. Phase has $\leq k$ distinct pages; new phase begins at request of $(k+1)$-st distinct. LRU: $\leq k$ misses/phase, because requested page survives until k other distinct pages requested. OPT: ≥ 1 miss/completed phase, because phase + first request of next phase contains $k+1$ distinct pages but cache holds k. Thus LRU $\leq k \cdot$OPT$+O(k)$; LRU is k-competitive.

2. Complexity / Asymptotics

$$\log n \ll \sqrt{n} \ll n \ll n \log n \ll n^2 \ll n^3 \ll 2^n \ll n! \ll n^n$$

$O(g): f \leq cg$ eventually. $\Omega(g): f \geq cg$. $\Theta(g) = O \cap \Omega$. $o(g): f/g \to 0$. $\omega(g): f/g \to \infty$. Stirling: $n! \sim \sqrt{2\pi n}(n/e)^n$, so $\log(n!) = \Theta(n \log n)$.

Useful sums: $\sum_{i=1}^n i = \frac{n(n+1)}{2} = \Theta(n^2)$, $\sum_{i=0}^{\log n} n = n \log n$, $\sum_{i \geq 0} 2^{-i} = 2$, $\sum_{h \geq 0} h 2^{-h} = 2$

Master / recurrence solver:

Master: $T(n) = aT(n/b) + f(n)$. Let $d = \log_b(a)$. Compare $f(n)$ with n^d.

f smaller: $f = O(n^{d-\varepsilon})$ $T = \Theta(n^d)$

same: $f = \Theta(n^d \log^k n)$ $T = \Theta(n^d \log^{k+1} n)$

f bigger: $f = \Omega(n^{d+\varepsilon})$ $T = \Theta(f)$ if regularity

Fast mental version if $f = n^k$:

$k < d$ leaves win $\Theta(n^d)$

$k = d$ tie/layers equal $\Theta(n^d \log n)$

$k > d$ root/work wins $\Theta(n^k)$

Non-equal splits: $T = \sum_i a_i T(b_i n) + \Theta(n^k)$. Find p s.t. $\sum_i a_i b_i^p = 1$.

$p > k$ leaves win $\Theta(n^p)$

$p = k$ tie $\Theta(n^p \log n)$

$p < k$ root/work wins $\Theta(n^k)$

Trap: if $\sum_i a_i b_i^p = 1$, plain cn^p proof has no slack; use $cn^p - dn^q$. Akra–Bazzi mini: $T(n) = \alpha T(an) + \beta T(bn) + \Theta(n^d)$. Find p: $\alpha a^p + \beta b^p = 1$. Then $p > d \Rightarrow \Theta(n^p)$, $p = d \Rightarrow \Theta(n^d \log n)$, $p < d \Rightarrow \Theta(n^d)$. Recurrences / runtimes:

binsearch	$T = T(n/2) + 1$ $\Theta(\log n)$
merge/maxsub	$T = 2T(n/2) + n$ $\Theta(n \log n)$
naive mm	$T = 8T(n/2) + n^2$ $\Theta(n^3)$
strassen	$T = 7T(n/2) + n^2$ $\Theta(n^{\log_2 7})$
heapify	$T \leq T(2n/3) + 1$ $O(\log n)$
qsort worst	$T = T(n-1) + n$ $\Theta(n^2)$
qsort bal	$T = 2T(n/2) + n$ $\Theta(n \log n)$
BF	$(V-1)E$ relax $\Theta(V \cdot E)$
BFS/DFS	visit V, scan E $\Theta(V+E)$

Master map: $T(n/2) + 1 \to \log n$; $T(n/2) + n \to n$; $2T(n/2) + 1 \to n$; $2T(n/2) + n \to n \log n$; $3T(n/2) + n \to n^{\log_2 3}$; $7T(n/2) + n^2 \to$ Strassen; $8T(n/2) + n^2 \to n^3$; $T(n-1) + n \to n^2$

3. Basic Data Structures

Stack / Queue / Linked List

Stack	push/pop/top	$O(1)$, array end, underflow if empty
Queue	enqueue/dequeue	$O(1)$ with circular array / deque
Linked list	search/insert/delete	search $O(n)$, known-node delete $O(1)$

Doubly linked list insert head: `x.next=head; if head!=NIL head.prev=x; head=x; x.prev=NIL`. Delete known node: reconnect `prev.next` and `next.prev`; finding node still $O(n)$.

Hash Tables

Idea: bucket $h(key) \bmod m$. Collisions: chaining / probing. Load factor $\alpha = n/m$.

Hash ops: insert/delete expected O(1), search expected O(1+alpha), all worst O(n); m=Theta(n) => alpha=Theta(1).

Expected total collisions: $X = \sum_{i<j} X_{ij}$, where $X_{ij} = 1[h(x_i) = h(x_j)]$. Universal hashing gives P[Xij=1]<=1/m, so E[X]=sum E[Xij] <= C(n,2)/m = n(n-1)/(2m)=Theta(n2/m). For one key: $\mathbb{E}[\#\text{collisions}] \leq (n-1)/m = \Theta(\alpha)$, hence expected search $O(1+\alpha)$. Traps: HashMap/HashSet $O(1)$ is expected/amortized, not worst-case. Bad hashing / all collisions $\Rightarrow O(n)$.

k-collision stencil: $X_S = 1$ if fixed k-set collides. $P[X_S = 1] \leq 1/m^{k-1}$. $E[X] = \sum_{|S|=k} E[X_S] \leq \binom{n}{k}/m^{k-1}$. If need existence prob: $P[X \geq 1] \leq E[X]$.

Binary Search Trees

Binary search (sorted array):

```
fn binary_search(a: Vec<_>,x){
  l=0; r=a.len();
  while l<r {
    m=(l+r)/2;
    if a[m]==x { return Some(m); }
    if a[m]<x  { l=m+1; }
    else       { r=m; }
  }
  None
}
//lower_bound = first >= x: first position where the value is >= x.
//upper_bound = first > x: first position where the value is > x.
```

BST property: BST property: all keys in L(x) < x.key; all keys in R(x) >= x.key.

BST ops search/insert/delete/min/max/succ/pred O(h); inorder Theta(n); balanced h=Theta(log n), degenerate h=Theta(n).

```
search(x,k):
  while x != NIL and x.key != k:
    if k < x.key: x = x.left
    else:         x = x.right
  return x

min(x): while x.left  != NIL: x = x.left;  return x
max(x): while x.right != NIL: x = x.right; return x

successor(x):
  if x.right != NIL: return min(x.right)
  y = x.parent
  while y != NIL and x == y.right:
    x = y
    y = y.parent
  return y
insert(T,z):
  y=NIL; x=T.root
  while x!=NIL:
    y=x
    if z.key<x.key: x=x.left else: x=x.right
  z.parent=y
  if y==NIL: T.root=z
  else if z.key<y.key: y.left=z
  else: y.right=z

transplant(T,u,v):
  if u.parent==NIL: T.root=v
  else if u==u.parent.left: u.parent.left=v
  else: u.parent.right=v
  if v!=NIL: v.parent=u.parent

delete(T,z):
  if z.left==NIL: transplant(T,z,z.right)
  else if z.right==NIL: transplant(T,z,z.left)
  else:
    y=min(z.right)
    if y.parent!=z:
      transplant(T,y,y.right)
      y.right=z.right; y.right.parent=y
    transplant(T,z,y);
    y.left=z.left; y.left.parent=y
```

Delete cases: no left, no right, or replace by successor. All $O(h)$.

Traversals:

```
inorder(x):  if x!=NIL: inorder(x.left); visit(x); inorder(x.right)
preorder(x): if x!=NIL: visit(x); preorder(x.left); preorder(x.right)
postorder(x):if x!=NIL: postorder(x.left); postorder(x.right); visit(x)
```

inorder(BST)=sorted order. Augmented BST / rank queries: Store size(x)=1+size(left)+size(right). Rank follows one root-to-leaf path, so $O(h)$. rank_<(x)=#keys $<$ x, rank_<=(x)=#keys $\leq$ x. Intervals: $[a,b] = \text{rank}_{<=}(b) - \text{rank}_{<}(a)$; $(a,b] = \text{rank}_{<=}(b) - \text{rank}_{<=}(a)$; $[a,b) = \text{rank}_{<}(b) - \text{rank}_{<}(a)$; $(a,b) = \text{rank}_{<}(b) - \text{rank}_{<=}(a)$.

4. Sorting Algorithms

Insertion Sort

In-place, stable. Invariant: before iteration j, prefix $A[0..j)$ sorted.

```
fn insertion_sort(a: &mut Vec<i32>) {
  for j in 1..a.len() {
    let key = a[j];
    let mut i = j;
    while i > 0 && a[i-1] > key {
      a[i] = a[i-1]; i -= 1;
    }
    a[i] = key;
  }
}
```

Best $\Theta(n)$, worst/avg $\Theta(n^2)$, space $O(1)$.

Merge Sort

Stable, simple version uses $\Theta(n)$ temp.

```
fn merge_sort(a,l,r){
  if l+1<= r { return; }
  let m=(l+r)/2;
  merge_sort(a,l,m);
  merge_sort(a,m,r);
  merge(a,l,m,r);
}

fn merge(a,l,m,r){
  let (mut i,mut j)=(l,m); let mut t=Vec::new();
  while i<m && j<r {
    if a[i] <= a[j] { t.push(a[i]); i+=1; }
    else { t.push(a[j]); j+=1; }
  }
  while i<m { t.push(a[i]); i+=1; }
  while j<r { t.push(a[j]); j+=1; }
  for k in 0..t.len(){ a[l+k]=t[k]; }
}
```

$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$.

Quicksort

In-place. Partition invariant: $A[l..i] \leq p$, $A[i+1..j] > p$, unknown after j, pivot at r.

```
fn partition(a,l,r)->usize{
  let p=a[r]; let mut i=l;
  for j in l..r {
    if a[j] <= p { a.swap(i,j); i+=1; }
  }
  a.swap(i,r); i
}

fn quicksort(a,l,r){
  if l>=r { return; }
  let q=partition(a,l,r);
  if q>0 { quicksort(a,l,q-1); }
  quicksort(a,q+1,r);
}
```

Balanced/expected randomized $\Theta(n \log n)$, worst $\Theta(n^2)$, stack expected $O(\log n)$ worst $O(n)$.

Counting Sort / Linear Sorting

Integer keys $0..k$, stable with prefix sums, not comparison-based.

```
fn counting_sort(a:&Vec<usize>, k:usize)->Vec<usize>{
  let mut c=vec![0;k+1]; let mut out=vec![0;a.len()];
  for &x in a { c[x]+=1; }
  for i in 1..=k { c[i]+=c[i-1]; }
  for idx in (0..a.len()).rev(){
    let x=a[idx]; c[x]-=1; out[c[x]]=x;
  }
  out
}
```

$\Theta(n+k)$ time/space. Stable with prefix sums. Needs bounded integer keys.

5. Heaps / Priority Queues / Heapsort

Max-Heapify

Assume children subtrees are heaps; only $A[i]$ may violate. 0-index:

```
fn max_heapify(a:&mut Vec<i32>, i:usize, heap_size:usize){
  let (l,r)=(2*i+1,2*i+1); let mut largest=i;
  if l<heap_size && a[l]>a[largest] { largest=l; }
  if r<heap_size && a[r]>a[largest] { largest=r; }
  if largest!=i { a.swap(i,largest); max_heapify(a,largest,heap_size); }
}
```

$O(height(i)) \leq O(\log n)$. Trap: during heapsort, heap_size $\neq$ a.len().

Build Max Heap

```
fn build_max_heap(a:&mut Vec<i32>){
  let n=a.len();
  for i in (0..(n/2)).rev() { max_heapify(a,i,n); }
}
```

Tight: nodes height $h \leq n/2^{h+1}$, cost $O(h)$, so $\sum_h nh/2^h = O(n)$; hence BuildHeap $\Theta(n)$, not $\Theta(n \log n)$.

Heapsort

```
fn heapsort(a:&mut Vec<i32>){
  build_max_heap(a); let mut h=a.len();
  while h>1 { a.swap(0,h-1); h-=1; max_heapify(a,0,h); }
}
```

Build $\Theta(n)$ + n extracts $O(\log n) = \Theta(n \log n)$, space $O(1)$, not stable.

Priority Queue

```
fn heap_extract_max(a:&mut Vec<i32>)->Option<i32>{
  if a.is_empty(){return None;}
  let ans=a[0]; let last=a.pop().unwrap();
  if !a.is_empty(){ a[0]=last; max_heapify(a,0,a.len()); }
  Some(ans)
}

fn heap_increase_key(a:&mut Vec<i32>, mut i:usize, key:i32){
  if key<a[i]{return;} a[i]=key;
  while i>0 { let p=(i-1)/2; if a[p]>=a[i]{break;} a.swap(i,p); i=p; }
}

fn max_heap_insert(a:&mut Vec<i32>, key:i32){
  a.push(i32::MIN); heap_increase_key(a,a.len()-1,key);
}
```

Std min-heap: `BinaryHeap<Reverse<(dist,node)>>`. Ops insert/extract/increase-key $O(\log n)$.

6. Divide & Conquer Algorithms

Karatsuba Multiplication

$$x = x_h B^m + x_l, \quad y = y_h B^m + y_l$$
$$a = x_h y_h, \quad b = x_l y_l, \quad c = (x_h + x_l)(y_h + y_l)$$
$$xy = aB^{2m} + (c - a - b)B^m + b$$

3 recursive multiplications.

3 recursive multiplications: $T(n) = 3T(n/2) + \Theta(n) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$.

Maximum Sum Subarray

Problem:
Input A[0..n) Output max sum contiguous nonempty subarray
D&C idea:
best subarray is:
- entirely left
- entirely right
- crosses middle

```
fn max_crossing(a,l,m,r){
  // left suffix [l,m)
  left_sum=-INF;
  sum=0;
  for i in (l..m).rev(){
    sum+=a[i];
    left_sum=max(left_sum,sum);
  }
  // right prefix [m,r)
  right_sum=-INF;
  sum=0;
  for j in m..r {
    sum+=a[j];
    right_sum=max(right_sum,sum);
  }
  left_sum+right_sum
}

fn max_subarray(a,l,r){
  if r-1==1 {return a[l]}
  m=(l+r)/2;
  left=max_subarray(a,l,m);
  right=max_subarray(a,m,r);
  cross=max_crossing(a,l,m,r);
  max(left,max(right,cross))
}
```

Complexity:
$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$

Linear Kadane:

```
fn kadane(a: &Vec<i32>) -> i32 {
    let mut best: i32 = a[0];
    let mut cur: i32 = a[0];

    for i in 1..a.len() {
        cur = a[i].max(cur + a[i]);
        best = best.max(cur);
    }

    best
}
```

Kadane invariant:

$$\text{cur}_i = \max(A_i, \text{cur}_{i-1} + A_i), \quad \text{best}_i = \max(\text{best}_{i-1}, \text{cur}_i)$$

Matrix Multiplication

Naive:

```
for i in 0..n {
    for j in 0..n {
        for k in 0..n {
            c[i][j] += a[i][k] * b[k][j];
        }
    }
}
```

Complexity:
$\Theta(n^3)$, space $\Theta(n^2)$

D&C block formula (8 mults): C11=A11B11+A12B21, C12=A11B12+A12B22, C21=A21B11+A22B21, C22=A21B12+A22B22. Hence $T(n) = 8T(n/2) + \Theta(n^2) = \Theta(n^3)$.

Strassen (7 mults): M1=(A11+A22)(B11+B22), M2=(A21+A22)B11, M3=A11(B12-B22), M4=A22(B21-B11), M5=(A11+A12)B22, M6=(A21-A11)(B11+B12), M7=(A12-A22)(B21+B22). C11=M1+M4-M5+M7, C12=M3+M5, C21=M2+M4, C22=M1-M2+M3+M6.

```
fn strassen(A,B,n){
  if n==1 { return A*B }
  split A,B into A11..A22,B11..B22;
  M1..M7 = strassen(formulas above);
  C11=M1+M4-M5+M7; C12=M3+M5;
  C21=M2+M4;       C22=M1-M2+M3+M6;
  combine(C11,C12,C21,C22)
}
```

$T(n) = 7T(n/2) + \Theta(n^2) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$.

7. Dynamic Programming

Rod Cutting

revenue$[\ell] =$ best revenue length ℓ, first$[\ell] =$ first cut. Rod cutting: revenue[0]=0; revenue[l]=max over 1<=c<=l of price[c]+revenue[l-c].

```
fn rod_cut(price:Vec<i32>, n:usize){
  revenue=vec![0;n+1]; first=vec![0;n+1];
  for length in 1..=n {revenue[length]=-INF;
    for cut in 1..=length {
      candidate=price[cut]+revenue[length-cut];
      if candidate>revenue[length] {
        revenue[length]=candidate; first[length]=cut
      }
    }
  }
  (revenue,first)
}
```

Reconstruct: output `first[length]`, set `length-=first[length]`. Runtime $\Theta(n^2)$.

Matrix Chain Multiplication

Matrix i: $p_i \times p_{i+1}$. $dp[i][j] =$ min cost for $A_i \cdots A_j$. MCM recurrence: dp[i][i]=0; dp[i][j]=min over i<=k<j of dp[i][k]+dp[k+1][j]+p[i]*p[k+1]*p[j+1].

```
fn matrix_chain(p:Vec<usize>){    // optimal parenthesization
  n=p.len()-1; dp=vec![vec![0;n];n]; split=vec![vec![0;n];n];
  for len in 2..=n {for i in 0..=n-len {
    j=i+len-1; dp[i][j]=INF;
    for k in i..j {
      cost=dp[i][k]+dp[k+1][j]+p[i]*p[k+1]*p[j+1];
      if cost<dp[i][j] {dp[i][j]=cost; split[i][j]=k}
    }
  }}
  (dp,split)
}
```

Increasing chain length. $\Theta(n^2)$ states $\times \Theta(n)$ splits $= \Theta(n^3)$.

Optimal BST

Sorted keys K_i with successful probs p_i; optional unsuccessful q_i. $e[i][j] =$ min expected cost for subtree $i..j$, $w[i][j] =$ prob mass, $root[i][j] =$ best root. OBST: e[i][i-1]=0, w[i][i-1]=q[i-1]; e[i][j]=min over root r in [i,j] of e[i][r-1]+e[r+1][j]+w[i][j].

```
fn optimal_bst(p,q,n){
  for i in 1..=n+1 {e[i][i-1]=0; w[i][i-1]=q[i-1]}
  for len in 1..=n {for i in 1..=n-len+1 {
    j=i+len-1; e[i][j]=INF; w[i][j]=w[i][j-1]+p[j]+q[j];
    for r in i..=j {
      cost=e[i][r-1]+e[r+1][j]+w[i][j];
      if cost<e[i][j] {e[i][j]=cost; root[i][j]=r}
    }
  }}
  (e,root)
}
```

Runtime $\Theta(n^3)$, space $\Theta(n^2)$.

Longest Common Subsequence

$dp[i][j] =$ LCS length of $A[0..i)$, $B[0..j)$. LCS: dp[0][*]=dp[*][0]=0. If A[i-1]==B[j-1]: dp[i][j]=1+dp[i-1][j-1]; else max(up,left).

```
fn lcs(a:Vec<char>, b:Vec<char>){
  n=a.len(); m=b.len(); dp=vec![vec![0;m+1];n+1];
  for i in 1..=n {for j in 1..=m {
    dp[i][j]=if a[i-1]==b[j-1] {1+dp[i-1][j-1]}
             else {dp[i-1][j].max(dp[i][j-1])}
  }}
  dp[n][m]
}

fn lcs_reconstruct(a:Vec<char>, b:Vec<char>, dp:Vec<Vec<usize>>){
  i=a.len(); j=b.len(); ans=vec![];
  while i>0 && j>0 {
    if a[i-1]==b[j-1] {ans.push(a[i-1]); i-=1; j-=1}
    else if dp[i-1][j]>=dp[i][j-1] {i-=1}
    else {j-=1}
  }
  ans.reverse(); ans
}
```

$\Theta(nm)$ time, $\Theta(nm)$ or rolling $\Theta(m)$ space. Equal $\to$ diagonal+1; unequal $\to$ max(up,left).

Knapsack 0/1

$dp[i][c] =$ best value using first i items, capacity C. 0/1 knapsack: dp[i][c]=max(skip dp[i-1][c], take v[i-1]+dp[i-1][c-w[i-1]]) if w[i-1]<=c.

```
fn knapsack(w:Vec<usize>, v:Vec<i32>, cap:usize){ // choose item at most once
  n=w.len(); dp=vec![vec![0;cap+1];n+1];
  for i in 1..=n {for c in 0..=cap {
    dp[i][c]=dp[i-1][c];
    if w[i-1]<=c {
      dp[i][c]=dp[i][c].max(v[i-1]+dp[i-1][c-w[i-1]])
```

```
    }
  }}
  dp[n][cap]
}
// 1D 0/1: capacity descending. Unbounded: capacity ascending.
```

$\Theta(nW)$ pseudo-polynomial.

Coin Change / Change-Making

Coin change: mincoins[0]=0; mincoins[x]=min over coin<=x of 1+mincoins[x-coin].

```
fn coin_change(coins:Vec<usize>, target:usize){ // unlimited coins
  mincoins=vec![INF;target+1]; mincoins[0]=0;   // min number of coins
  for amount in 1..=target {for coin in coins {
    if coin<=amount {
      mincoins[amount]=mincoins[amount].min(1+mincoins[amount-coin])
    }
  }}
  mincoins[target]
}
```

Runtime $\Theta(target \cdot \#coins)$.

Longest Increasing Subsequence

Quadratic: LIS $O(n^2)$: $dp[i] = 1 + \max\{dp[j] : j < i, a[j] < a[i]\}$; answer$=\max dp$.

```
fn lis_n2(a:Vec<i32>){
  dp=vec![1;a.len()];
  for i in 0..a.len(){for j in 0..i{
    if a[j]<a[i]{dp[i]=dp[i].max(1+dp[j])}
  }}
  max(dp)
}
fn lis_nlogn(a:Vec<i32>){ // tails[len-1] = min tail
  tails=vec![];
  for x in a {
    l=0; r=tails.len();
    while l<r {m=(l+r)/2; if tails[m]<x {l=m+1}else{r=m}}
    if l==tails.len(){tails.push(x)}else{tails[l]=x}
  }
  tails.len()
}
```

DAG Longest Path

Input `adj: WGraph`. Topological order; max-relaxation; negative weights OK because no cycles.

```
fn dag_longest(adj:WGraph, source:usize){
  n=adj.len(); dist=vec![-INF;n]; parent=vec![None;n]; dist[source]=0;
  for u in topo(adj) {if dist[u]!=-INF {for (v,w) in adj[u] {
    if dist[u]+w>dist[v] {dist[v]=dist[u]+w; parent[v]=Some(u)}
  }}}
  dist
}
```

Runtime $\Theta(V + E)$.

8. Graph Representation / Types

Use these names everywhere:

```
type Graph  = Vec<Vec<usize>>;         // adj[u] = neighbors v
type WGraph = Vec<Vec<(usize,i32)>>;   // adj[u] = (v,weight)
struct Edge { u:usize, v:usize, w:i32 }
type EdgeList = Vec<Edge>;
```

adj list	space $\Theta(V+E)$	scan neighbors $\Theta(\deg u)$
adj matrix	space $\Theta(V^2)$	edge lookup $\Theta(1)$, scan $\Theta(V)$
edge list	space $\Theta(E)$	Bellman-Ford/Kruskal scans all edges

9. BFS

Input `adj: Graph`. Unweighted shortest paths / levels / components / bipartite. Runtime $\Theta(V + E)$.

```
fn bfs(adj:Graph, source:usize){
  n=adj.len(); // shortest path in unweighted graph
  dist=vec![INF;n]; // levels, components
  parent=vec![None;n];
  queue=VecDeque::new();
  dist[source]=0;
  queue.push_back(source);

  while let Some(u)=queue.pop_front(){
    for v in adj[u] {
      if dist[v]==INF {
        dist[v]=dist[u]+1;
        parent[v]=Some(u);
        queue.push_back(v);
      }
    }
  }
  (dist,parent)
}
```

```
fn bipartite(adj:Graph){          // 2-coloring
  n=adj.len(); color=vec![-1;n];  // false iff odd cycle
  queue=VecDeque::new();
  for s in 0..n {
    if color[s]!=-1 {continue}
    color[s]=0; queue.push_back(s);
    while let Some(u)=queue.pop_front(){
      for v in adj[u] {
        if color[v]==-1 {
          color[v]=1-color[u];
          queue.push_back(v);
        }
        else if color[v]==color[u] {return false}
      }
    }
  }
  true
}
```

<u>Invariant:</u> queue processed in nondecreasing distance. First discovery gives shortest unweighted distance. If a shorter path to v existed, its predecessor would have been processed earlier, so v would already be discovered.
<u>Uses:</u> unweighted shortest path, levels, connected components, bipartite test.
<u>Path reconstruction:</u> follow parent$[t]$ backward until source, then reverse.

10. DFS / Topological Sort / SCC

Input `adj: Graph`. Colors: WHITE/GRAY/BLACK. Runtime $\Theta(V + E)$.

```
fn dfs(adj:Graph){  // topo, cycle detection, SCC
  n=adj.len(); color=vec![WHITE;n]; parent=vec![None;n]; time=0;
  fn visit(u:usize){
    time+=1; discover[u]=time; color[u]=GRAY;
    for v in adj[u] {
      if color[v]==WHITE {parent[v]=Some(u); visit(v)}
    }
    color[u]=BLACK; time+=1; finish[u]=time;
  }
  for u in 0..n {if color[u]==WHITE {visit(u)}}
}
```

Directed edges: tree = white target; back = gray ancestor; forward/cross = black. Undirected: only tree/back; cycle test = visited neighbor $\neq$ parent. DFS times: d[u]<d[v]<f[v]<f[u] => descendant/tree/forward; d[v]<d[u]<f[u]<f[v] => back to ancestor/cycle; disjoint intervals => cross. Directed cycle detection: edge to GRAY vertex $\Leftrightarrow$ back edge $\Leftrightarrow$ directed cycle. Edge to BLACK does not prove a cycle.

Topological Sort

DAG order: every edge $u \to v$ has u before v. DFS = decreasing finish times.

```
fn topo_dfs(adj:Graph){
  n=adj.len(); seen=vec![false;n]; order=vec![];
  fn visit(u:usize){
    seen[u]=true;
    for v in adj[u]{if !seen[v]{visit(v)}}
    order.push(u)
  }
  for u in 0..n {if !seen[u]{visit(u)}}
  order.reverse(); order
}
```

```
fn topo_kahn(adj:Graph){
  n=adj.len(); indeg=vec![0;n];
  queue=VecDeque::new(); order=vec![];
  for u in 0..n {for v in adj[u]{indeg[v]+=1}}
  for u in 0..n {if indeg[u]==0 {queue.push_back(u)}}
  while let Some(u)=queue.pop_front(){
    order.push(u);
    for v in adj[u]{
      indeg[v]-=1;
      if indeg[v]==0{queue.push_back(v)}
    }
  }
  if order.len()==n {Some(order)} else {None}
}
```

Kahn returns `None` iff not all vertices are output.

Strongly Connected Components

SCC: $u \leftrightsquigarrow v$. Kosaraju = DFS finish order, reverse graph, DFS in decreasing finish order.

```
fn kosaraju(adj:Graph){   // SCC. DFS order -> reverse graph -> DFS
  order=dfs_finish_order(adj); rev=reverse_graph(adj);
  seen=vec![false;n]; comps=[];
  for source in order.rev() {
    if seen[source]{continue}
    comp=[]; stack=[source]; seen[source]=true;
    while let Some(u)=stack.pop(){
      comp.push(u);
      for v in rev[u]{
        if !seen[v]{seen[v]=true; stack.push(v)}
      }
    }
    comps.push(comp)
  }
  comps
}
```

Runtime $\Theta(V + E)$.

11. Union-Find / Disjoint Sets

Forest of trees. parent$[x]$ points upward. Root is representative. rank$[x]$ upper-bounds height.

```
struct DSU {
  parent: Vec<usize>,
  rank: Vec<usize>,
}
impl DSU{
  fn new(n){DSU{parent:(0..n).collect(),
    rank:vec![0;n]}}

  fn find(&mut self,x){
    if self.parent[x]!=x {
      self.parent[x]=self.find(self.parent[x])
    }
    self.parent[x]
  }
```

```
  fn union(&mut self,x,y){
    let (mut rx,mut ry)=(self.find(x),self.find(y));
    if rx==ry {return false}
    if self.rank[rx] < self.rank[ry]
      { swap(rx,ry) }
    self.parent[ry]=rx;
    if self.rank[rx]==self.rank[ry] {
      self.rank[rx]+=1;
    }
    true
  }
}
```

Complexities:

- linked list weighted union: $O(m + n \log n)$
- forest + union by rank + path compression: $O(m\alpha(n))$

$\alpha(n) \leq 5$ in practice.

```
//Connected components:
let mut dsu: DSU = DSU::new(n);
for (u, v) in edges {
    dsu.union(u, v);
}
if dsu.find(a) == dsu.find(b) {
    // same component
}
```

```
//Cycle detection (undirected):
let mut dsu: DSU = DSU::new(n);
for (u, v) in edges {
    if !dsu.union(u, v) {
        // cycle
    }
}
```

12. Minimum Spanning Trees

Chooser

sparse graph	Kruskal often easiest
dense graph	Prim often easier
disconnected graph	forest, no MST

tree: connected + acyclic connected graph: tree $\Leftrightarrow$ |E|=|V|-1

MST: spanning tree with min total edge weight

Cut / Cycle Properties

Cut $(S, V - S)$: edge crosses cut iff one endpoint in S, one outside. Light edge = minimum-weight crossing edge.

<u>Cut property:</u> if cut respects chosen edge set A, then a light crossing edge is safe.

<u>Cycle property:</u> for any cycle, a heaviest edge on that cycle is excluded from some MST. If weights distinct, the heaviest edge is in no MST.

Kruskal

Data structures:

```
struct Edge {
    u,
    v,
    w: i32,
}

let mut edges: Vec<Edge> = ...;
let mut dsu: DSU = DSU::new(n);
let mut mst: Vec<Edge> = Vec::new();
let mut total: i32 = 0;
```

Algorithm:

```
fn kruskal(n,edges: Vec<Edge>){ // MST
  sort_by_weight(edges); dsu=DSU::new(n); total=0; mst=[];
  for e in edges {              // sort edges + DSU
    if dsu.union(e.u,e.v) {
      total+=e.w; mst.push(e);
      if mst.len()==n-1 {break}
    }
  }
  (total,mst)
}
```

Complexity: sort: $O(E \log E)$ DSU: $O(E\alpha(V))$ total: $O(E \log E)$

<u>Output condition:</u> if `mst.len()==n-1`, it is an MST; otherwise graph disconnected and result is a forest.

Traps:

- If graph disconnected $\longrightarrow$ MST does not exist; returns forest.
- Need sort by weight increasing.
- Skip edge if endpoints already connected.

Prim

```
let mut adj: Vec<Vec<(usize, i32)>> = vec![Vec::new(); n];
let mut used: Vec<bool> = vec![false; n];
let mut heap: BinaryHeap<Reverse<(i32, usize, usize)>> = BinaryHeap::new();
// (weight, vertex, parent)
fn prim(adj,s){                      // MST
  heap.push((0,s)); total=0; cnt=0;   // min crossing edge
  while let Some((w,u))=heap.pop_min(){
    if used[u] {continue}
    used[u]=true; total+=w; cnt+=1;
    for (v,w2) in adj[u] {
      if !used[v] {heap.push((w2,v))}
    }
  }
  if cnt!=n {/* disconnected */}
  total
}
```

Complexity:

- Binary heap lazy: $O(E \log E)$; for simple graphs $E \leq V^2$, so $O(E \log V)$.

<u>Output condition:</u> if `cnt==n`, `total` is MST weight; otherwise graph disconnected.

MST Reduction Stencils

<u>Roads+airports:</u> add supernode s; edge s--city cost=airport; old road edges unchanged; MST in augmented graph. <u>Add one new vertex given old MST T:</u> run MST on T plus new incident edges; old non-tree edges still unnecessary by cut/cycle property. <u>Points/routes:</u> complete graph on points, edge cost=squared distance; run Kruskal/Prim. $E = \Theta(n^2)$, time $O(n^2 \log n)$.

13. Shortest Paths

Chooser

unweighted	BFS
DAG	topo DP
weights ≥ 0	Dijkstra
negative edges	Bellman-Ford

Relaxation

Relax edge (u,v,w): if dist[u]!=INF and dist[v] > dist[u]+w, set dist[v]=dist[u]+w and parent[v]=u.

Dijkstra

```
fn dijkstra(adj:WGraph, source:usize){ // all weights >= 0
  let n=adj.len();
  let mut dist=vec![INF;n];
  let mut parent=vec![None;n];
  let mut heap=MinHeap::<(i32,usize)>::new();

  dist[source]=0;
  heap.push((0,source));

  while let Some((du,u))=heap.pop(){
    if du!=dist[u] {continue} // stale heap entry

    for (v,w) in adj[u] {
      if dist[v]>du+w {
        dist[v]=du+w;
        parent[v]=Some(u);
        heap.push((dist[v],v));
      }
    }
  }
  dist
}
```

Runtime $O((V + E) \log V)$. Requires all weights ≥ 0; negative edges break it.

Bellman-Ford

Input `edges: EdgeList`. Negative edges OK; detects reachable negative cycles.

```
fn bellman_ford(n, edges, source){
  dist=vec![INF;n]; parent=vec![None;n]; dist[source]=0;
  for _ in 0..n-1 { changed=false;
    for e in edges {
      if dist[e.u]!=INF && dist[e.v]>dist[e.u]+e.w {
        dist[e.v]=dist[e.u]+e.w; parent[e.v]=Some(e.u);
        changed=true;
      }
    }
    if !changed {break}
  }
  for e in edges {
    if dist[e.u]!=INF && dist[e.v]>dist[e.u]+e.w {
      return None; // reachable negative cycle
    }
  }
  Some(dist)
}
```

Runtime $\Theta(VE)$. Returns `None` iff a reachable negative cycle is detected. <u>Negative cycle not reachable from source:</u> Bellman-Ford still returns shortest paths. Only improvements after pass V reachable from source matter.

<u>Weight transform trap:</u> $w'(e) = a\,w(e)$ with $a > 0$ preserves shortest paths. $w'(e) = w(e) + a$ does not preserve shortest paths when candidate paths use different numbers of edges.

Shortest Paths in DAG

```
fn dag_shortest(adj:WGraph, source:usize){
  let n=adj.len();
  let mut dist=vec![INF;n];
  let mut parent=vec![None;n];

  dist[source]=0;

  for u in topo(adj) {
    if dist[u]==INF {continue}
    for (v,w) in adj[u] {    // topological order.
      if dist[v]>dist[u]+w { // one relaxation per edge
        dist[v]=dist[u]+w;   // negative weights ok
        parent[v]=Some(u);
      }
    }
  }
  dist
}
```

Runtime $\Theta(V + E)$.

Time-Dependent / Ferry Shortest Path

$dist[u]$ = earliest arrival. Road relax: $dist[v] = \min(dist[v], dist[u] + w)$. Ferry/event (u, v, dep, arr) usable iff $dist[u] \leq dep$, then relax $dist[v] = \min(dist[v], arr)$.

Use PQ by earliest arrival if time is FIFO/nonnegative; otherwise expand state/event graph.

14. Flow Networks

Directed $G = (V, E)$, source s, sink t, capacities $c(u, v) \geq 0$. Flow constraints: 0<=f(u,v)<=c(u,v); conservation for u!=s,t: inflow=outflow; |f|=out(s)-in(s). Augmenting path: $s \longrightarrow t$ path in residual graph; bottleneck $= \min c_f(e)$ on path. <u>Flow reduction recipe:</u> selected object = unit flow; capacities = usage limits; source/sink = constraints; value = objective; integral caps => integral solution.

Ford-Fulkerson / Edmonds-Karp

Types: `cap[u][v]` residual capacity matrix; `adj: Graph` stores forward+reverse residual neighbors.

```
fn add_edge(u:usize,v:usize,c:i32){
  adj[u].push(v); adj[v].push(u); cap[u][v]+=c
}
fn bfs_path(source:usize, sink:usize){
  parent.fill(None); parent[source]=Some(source);
  queue.push_back((source,INF));
  while let Some((u,flow))=queue.pop_front(){
    for v in adj[u]{
      if parent[v].is_none() && cap[u][v]>0 {
        parent[v]=Some(u);
        pushed=flow.min(cap[u][v]);
        if v==sink {return pushed}
        queue.push_back((v,pushed))
      }
    }
  }
  0
}
```

```
fn max_flow(source:usize, sink:usize){ // max flow
  flow=0;                 // augment while path exists
  loop {
    pushed=bfs_path(source,sink);
    if pushed==0 {break}
    flow+=pushed;
    v=sink;
    while v!=source {
      u=parent[v].unwrap();
      cap[u][v]-=pushed; cap[v][u]+=pushed;
      v=u
    }
  }
  flow
}
```

Ford-Fulkerson integer caps: $O(E|f^*|)$. Edmonds-Karp BFS: $O(VE^2)$. Final residual graph gives min cut by residual reachability from s.

Max-Flow Min-Cut / Reductions

Max-flow min-cut: $\max |f| = \min c(S, V \setminus S), s \in S, t \notin S$. Optimality certificate: flow value F and cut capacity F.

Bipartite matching: source$\longrightarrow L$ cap 1; $L \longrightarrow R$ cap 1 for original edges; $R \longrightarrow$sink cap 1. Flow-1 $L \longrightarrow R$ edges are matched pairs.

Capacitated assignment: $s \longrightarrow$ person cap 1; person$\longrightarrow$ acceptable activity cap 1; activity$\longrightarrow t$ cap capacity. Feasible iff maxflow = #people. Integral capacities give integral assignment.

```
source=0; left0=1; right0=1+n_left; sink=1+n_left+n_right;
for l in 0..n_left {add_edge(source,left0+l,1)}
for (l,r) in bip_edges {add_edge(left0+l,right0+r,1)}
for r in 0..n_right {add_edge(right0+r,sink,1)}
// flow-1 edge left0+l -> right0+r = matched pair
```

Edge-disjoint paths: all edges cap 1. Vertex-disjoint: split v into $v_{in} \longrightarrow v_{out}$ cap 1; original $u \longrightarrow v$ becomes $u_{out} \longrightarrow v_{in}$ cap INF. <u>Grid domino:</u> checkerboard cells L/R, edge for adjacency, max dominoes = max bipartite matching. <u>Path $x \longrightarrow y$ through z:</u> need two internally vertex-disjoint paths $z \rightsquigarrow x$ and $z \rightsquigarrow y$; split vertices cap 1, add $x, y \rightarrow t$ cap 1, ask maxflow $z \longrightarrow t$ value 2. <u>Special edge $e = ab$ used:</u> try orientations $\cdots a - b \cdots$ and $\cdots b - a \cdots$; remove e, enforce disjoint prefix/suffix with unit capacities.

15. Randomized / Probabilistic Analysis

Indicator: $X_A = 1[A], \mathbb{E}[X_A] = \Pr(A)$. Linearity: $E[\Sigma_i X_i] = \Sigma_i E[X_i]$. No independence needed.
Hiring: $P(i \text{ new best}) = 1/i$, so $E[\#hires] = \Sigma_i 1/i = H_n = \Theta(\log n)$. Hat-check: $P(i \text{ own hat}) = 1/n$, so $E[\#fixed] = 1$.

Randomized quicksort: random pivot avoids input-dependent bad order; expected $\Theta(n \log n)$, worst still $\Theta(n^2)$.

<u>Birthday:</u> $\Pr[\text{all different among } q \text{ into } m] = \Pi_{i=0}^{q-1}(1 - i/m) \approx e^{-q(q-1)/(2m)}$; for 365 days, $q = 23$ gives match prob $> 1/2$. Integral bound for decreasing positive f: $\Sigma_{k=a+1}^{b} f(k) \leq \int_a^b f(x)\,dx$.

16. Online Algorithms / Competitive Analysis / Experts

Online: decide before future known. Competitive ratio: $ALG(I) \leq r\,OPT(I) + c$. <u>Ski rental:</u> rent 1/day, buy B. Rent until paid B, then buy. If $T \leq B$: ALG=T=OPT. If $T > B$: ALG<=B+B=2B=2OPT. Lower bound: adversary stops just before/after purchase; deterministic CR=2.

Caching MC traps

LRU evicts least-recently requested page. LFU evicts smallest request count; tie usually LRU. Neither has pointwise fewer misses than the other on every sequence. LRU is k-competitive for cache size k; no deterministic paging algorithm beats k in worst-case competitive ratio.

Experts / Weighted Majority

n experts, rounds $t = 1..T$. Expert i predicts; loss $\ell_{i,t} \in \{0, 1\}$. Weights $w_i = 1$ initially. Let $W = \Sigma_i w_i$.

```
for t in 1..=T {
  predict weighted majority of experts;
  observe true outcome;
  for expert i wrong this round {
    w[i] *= beta;        // 0 < beta < 1, often beta=1/2
  }
}
```

Weighted prediction: choose label with larger total weight. Potential: $W_t = \Sigma_i w_i^{(t)}$. If ALG makes mistake, wrong-weight $\geq W_t/2$, so $W_{t+1} \leq W_t/2 \cdot \beta + W_t/2 = W_t(1 + \beta)/2$. If best expert has m^* mistakes, its weight is β^{m^*}, hence $\beta^{m^*} \leq W_T \leq n((1 + \beta)/2)^M$. For $\beta = 1/2$: $M \leq 2.41(m^* + \log_2 n)$. General randomized Hedge: choose expert with prob w_i/W; update $w_i \leftarrow w_i e^{-\eta\ell_{i,t}}$; regret $O(\sqrt{T \log n})$.

Prophet Inequality

Independent nonnegative X_i, known distributions. $M = \max_i X_i, z = E[M]$, threshold $\tau = z/2$. Accept first $X_i \geq \tau$. Guarantee $E[ALG] \geq z/2$. Proof stencil: $q = P[\forall i : X_i < \tau]$, $b_i = E[(X_i - \tau)^+]$. Value from threshold part $\geq \tau(1 - q)$; overshoot part $\geq q\,\Sigma_i b_i$; combine to get $\geq z/2$.

Secretary: random order; observe first n/e candidates, remember best rank seen, then hire first later candidate better than all observed.

$$P(\text{success}) \approx 1/e$$

17. WORKED EXAM PROBLEMS

Representative problems from exercise sets / exams. Question $\longrightarrow$ exact answer skeleton.

Q1 (Ex1). Prove Linear-Search correct with a loop invariant.
Inv.: at start of iteration i, $A[j] \neq v$ for all $j < i$. Init: $i = 1$: empty range. Maint: if invariant true and $A[i] \neq v$, then $A[j] \neq v$ for all $j < i + 1$. Term: either returns i with $A[i] = v$, or $i = n + 1$ so $A[j] \neq v$ for all j and returns NIL.

Q2 (Ex1). Show $(n + a)^b = \Theta(n^b)$ for constants $a, b > 0$.
For $n \geq 2|a|$: $\frac{1}{2}n \leq n + a \leq 2n$, hence $(\frac{1}{2})^b n^b \leq (n + a)^b \leq 2^b n^b$. Both sides are constants times n^b; therefore $(n + a)^b = \Theta(n^b)$.

Q3 (Ex2). Solve $T(n) = 8T(n/2) + \Theta(n^2)$ by recursion tree.
Level i: 8^i nodes, each cost $c(n/2^i)^2$, total $cn^2 2^i$. Sum $\Sigma_{i=0}^{\lg n} cn^2 2^i = \Theta(n^2 \cdot n) = \Theta(n^3)$. Leaves: $n^{\lg 8} = n^3$. Hence $T(n) = \Theta(n^3)$.

Q4 (Ex2). Recursive insertion sort recurrence.
$T(n) = T(n-1) + \Theta(n)$. Unroll: $T(n) = c\,\Sigma_{i=1}^{n} i = c\,n(n+1)/2 = \Theta(n^2)$.

Q5 (Ex6). n restaurants on a line, positions M_i, profit p_i, any two at distance $\geq k$. Max profit.
Sort by position. State $dp[i]$ = best profit using restaurants $i..n$. Let $next(i)$ = first $j > i$ with $M_j \geq M_i + k$. Base $dp[n+1] = 0$. Recurrence $dp[i] = \max(dp[i+1], p_i + dp[next(i)])$. Order $i = n..1$. Ans: $dp[1]$. Runtime $O(n \log n)$ with binary search, $O(n^2)$ if scanning. Correctness: optimum either skips i or takes i; after taking i, remaining choices start at $next(i)$.

Q5b (Ex6). Restaurant leftmost-open variant.
Sort positions. $c[i]$ = best total profit if i is leftmost opened. Then $c[i] = p_i + \max\{c[j] : j > i \wedge M_j \geq M_i + k\}$. Ans: $\max_i c[i]$. Bottom-up $O(n^2)$; with binary search/suffix maxima $O(n \log n)$. Fixing the leftmost open restaurant leaves an independent feasible suffix.

Q6 (Ex5). $O(\log n)$ test whether sorted $A[1..n]$ contains k.
Binary search. Invariant: if k occurs, then $k \in A[l..r)$. Compare k with $A[mid]$ and keep only the half that can contain k. Sortedness preserves invariant. Termination: range empty $\Rightarrow$ absent, or equality found. Runtime $T(n) = T(n/2) + O(1) = O(\log n)$.

Q7 (Ex9). Maximum bipartite matching as a flow.
Build network: add $s \rightarrow L$, original edges $L \rightarrow R$, $R \rightarrow t$, all capacities 1. Integral max-flow gives integral solution. Capacity 1 forces each left/right vertex used at most once. Unit-flow edge $l \rightarrow r$ is a matched pair; every matching gives same-size flow. Thus max flow = max matching.

Q9 (Ex12). What does Lomuto Partition return on an all-equal array, and how to fix it?
With condition $\leq pivot$, every element is moved left, so $q = r$. Split sizes $n - 1$ and 0, hence $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$. Fix: 3-way partition into $<, =, >$ pivot, or split equal keys evenly / return middle of equal block.

Q10 (L19 exam). Ski resort: lifts/slopes have signed altitude changes. Verify every closed tour returns to start with net change 0.
Build digraph: stations = vertices, lift/slope = directed edge weighted by altitude change. Bad tour $\Leftrightarrow$ directed cycle with nonzero total weight. Run Bellman-Ford from super-source on weights w and on $-w$: a positive cycle in w is negative in $-w$; a negative cycle in w is detected in w. Relaxation on pass V in either run $\Rightarrow$ nonzero cycle exists. Runtime $O(VE)$.

Q11 (L19). (a) Min crossing edge of any cut is in the MST. (b) Max edge of any cycle is not.
Assume distinct weights. (a) Cut property: if lightest crossing edge e absent, add e to MST; cycle contains another crossing edge f with $w(f) > w(e)$; swap $f \mapsto e$ gives cheaper tree, contradiction. (b) If heaviest cycle edge e in MST, remove it; the cycle has lighter edge f crossing the induced cut; swap $e \mapsto f$ gives cheaper tree, contradiction.

Q12. Travel passes DP.
Travel days $d_1 < \cdots < d_n$, pass lengths $L \in \{1, 7, 30\}$, costs c_L. State $dp[i]$ = min cost to cover $d_i..d_n$. Base $dp[n + 1] = 0$. Let $next(i, L)$ = first j with $d_j \geq d_i + L$. Recurrence $dp[i] = \min_{L\in\{1,7,30\}}(c_L + dp[next(i, L)])$. Order $i = n..1$. Ans: $dp[1]$. Runtime $O(n \log n)$ with lower_bound. Trap: pass covers days $< d_i + L$, not $\leq d_i + L$.

Q13. Weighted interval scheduling.
Sort intervals by finish time. $p(i)$ = last interval finishing $\leq start_i$. State $dp[i]$ = best weight using first i intervals. Base $dp[0] = 0$. Recurrence $dp[i] = \max(dp[i-1], w_i + dp[p(i)])$. Ans: $dp[n]$. Runtime $O(n \log n)$ with binary search for $p(i)$. Correctness: optimum either excludes interval i or includes it, then previous chosen interval must be $\leq p(i)$.

Q14. Interval scheduling greedy.
Sort by increasing finish time. Take interval if $start \geq last_finish$. Proof: let e be earliest finishing interval. For any optimum O, replace the first interval of O by e; finish time does not increase, so all later intervals remain feasible. Therefore some optimum contains e. Induct on remaining intervals.

Q15. DP design answer skeleton.
State $dp[s]$ = optimum value for subproblem s. Base: smallest/empty cases. Recurrence $dp[s] = \text{opt}_{c\in valid(s)} F(s, c, dp[next(s, c)])$. Order: dependencies first. Ans: explicitly write returned state. Runtime = #states $\times$ transition cost. Correctness: induction in fill order; recurrence enumerates all valid choices; dependencies optimal by IH.

Q16a. Parametric search / split array into k contiguous parts minimizing max load.
Bounds $L = \max_i a_i$, $R = \Sigma_i a_i$. feasible(M): scan left-to-right, start new part when current sum $+a_i > M$. Count parts. If count $\leq k$, M feasible. Binary search smallest feasible M. Correctness: greedy uses minimum possible number of parts for fixed M; feasibility monotone. Runtime $O(n \log(\Sigma a_i))$.

Q16b. Forbidden mutual reachability pairs in directed graph.
Compute SCCs in $O(V + E)$. For each forbidden pair (u, v), reject iff $comp[u] = comp[v]$. If same SCC, then $u \rightsquigarrow v$ and $v \rightsquigarrow u$. If mutually reachable, they must be in same SCC. Thus all constraints checked exactly.

Q16c. Capacitated activity assignment.
Network: $s \longrightarrow$ each student cap 1; student$\longrightarrow$ acceptable activity cap 1; activity$\longrightarrow t$ cap number of spots. Run maxflow. Ans: feasible iff flow value = #students. Correctness: integral unit leaving each student chooses one acceptable activity; activity capacity enforces spots; any assignment gives same flow.

Q16d. Binary tree diameter / longest path.
Postorder. Empty height $= -1$. For node u: $h(u) = 1 + \max(h_L, h_R)$ and $diam(u) = \max(diam_L, diam_R, h_L + h_R + 2)$. Ans: $diam(root)$. Correctness: every longest path either stays inside one child subtree or crosses through u using deepest node on each side.

Q16e. Connected graph with n vertices and n edges: count spanning trees.
Such graph has exactly one cycle. Find the cycle by DFS back edge or by repeatedly deleting degree-1 leaves. If cycle length is C, answer is C. Reason: a spanning tree has $n - 1$ edges, so delete one edge. Deleting a cycle edge keeps connected and removes the only cycle; deleting a non-cycle edge disconnects.

Q16f. Coin-row 2-player minimax DP.
$dp[i][j]$ = max value current player can guarantee from coins $i..j$. Base $dp[i][i] = a_i$. Recurrence $dp[i][j] = \max(a_i + \min(dp[i+2][j], dp[i+1][j-1]), a_j + \min(dp[i+1][j-1], dp[i][j-2]))$. Opponent chooses the branch minimizing our future gain. Fill by interval length.

Q17. Competitive-ratio template.
Define OPT(I). Compute ALG(I). Split into exhaustive cases. In each case prove ALG$(I) \leq r\,OPT(I) + c$. Take worst case. Conclusion: algorithm is r-competitive. Lower bound: adversary chooses future after seeing deterministic algorithm action.

Q18. Secretary problem.
Random order. Reject first $\lfloor n/e \rfloor$ candidates, remember best seen, then hire first later candidate better than all previous. Success probability tends to $1/e$. Reason: best candidate must appear after sample, and best candidate before it must lie inside sample.

Q19. Online bipartite matching.
Left side known, right vertices arrive online. Greedy: match arriving vertex to any free neighbor. Each greedy edge can be charged by at most two OPT edges, so ALG $\geq \frac{1}{2}$ OPT. Ranking: random rank/permutation on left vertices; match arriving vertex to lowest-ranked free neighbor. Expected $(1 - 1/e)$OPT.

Q20. Prophet inequality threshold algorithm.
Known independent values X_i. Let $M = \max_i X_i$, $z = E[M]$, threshold $\tau = z/2$. Accept first value at least τ. Guarantee $E[ALG] \geq z/2$. Write proof with $q = P[\text{no value reaches } \tau]$ and overshoots $(X_i - \tau)^+$; threshold contribution plus overshoot contribution gives half of prophet value.

Q21. Inorder+postorder tree reconstruction.
root=last(postorder). Split inorder at root into left/right. Sizes determine postorder left/right blocks. Recurse. Correctness by induction on subtree size. Runtime $O(n)$ with key$\longrightarrow$inorder-index map.

Q22. Longest path passing through root.
Compute subtree heights. Answer $h(left) + h(right) + 2$ with empty height -1. Correct because path through root must be deepest node in left side + root + deepest node in right side.

Q23. Domino/grid matching.
Checkerboard color cells. Put black cells in L, white cells in R, edge iff adjacent. A domino is one matched edge; no cell reused because matching. Max domino placement = max bipartite matching.